VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
Bandaru Rushindra	24MIM10145
Slot	**Date of Submission**
D11	31/03/2026

1. Introduction

Most of the software we use every day is not from one company, but many companies working together to build software using open source code from all over the world.

For example, the Linux Kernel, the Apache HTTP Server, and Python have been used in many different systems: from your web site to the servers that run them or your mobile phones. I did not think about how these tools were created until I had to do this project. I only used them.

This project made me think about the software that I use: why was this software made, who works on it, and why do people give away their time and effort for free?

As part of this project, I have to find one open-source software project, and then study it. I will look at what it does, but I will also learn something about its history, its license, and how its use has affected its creator. At first, I thought it would be easy, but as I began to read about it; I found it very difficult to understand all the licenses and all the people associated with this project. There are a lot of people involved with this type of project.

Another aspect of this project involves creating shell scripts on Linux platform. Before writing a complete script, I had already used some basic commands within the Linux environment. In the beginning, I made some beginner-level mistakes, forgetting to set permissions on my scripts or mistyping the commands when executing them. Once I figured out my errors, it became much easier to use and actually create a successful script.

Overall, this project has been both theoretical in nature as well as practical. I have had an opportunity to not only learn the theory behind scripting, but also gain skill in actually doing it.

2. Choosing Your Software

Pick one open-source project from the list below, or propose your own (subject to instructor approval). Your choice will be the subject of your entire report and all five shell scripts.

Approved software options

Software	Category	License	Why it's interesting
Linux Kernel	Operating System	`GPL v2`	*The foundation everything else runs on*
Apache HTTP Server	Web Server	`Apache 2.0`	*Powers ~30% of all websites globally*
MySQL	Database	`GPL v2 / Commercial`	*A license with a dual-model story to tell*
Firefox	Web Browser	`MPL 2.0`	*A nonprofit fighting for an open web*
VLC Media Player	Multimedia	`LGPL / GPL`	*Plays anything — built by students in Paris*
LibreOffice	Office Suite	`MPL 2.0`	*Born from a community fork — a real lesson*
Python	Programming Language	`PSF License`	*A language shaped entirely by community*
Git	Version Control	`GPL v2`	*The tool Linus built when proprietary failed him*

Your choice	GIT _____

3. Project Report Structure

Part A — Origin and Philosophy (Units 1 & 2)

A1. The problem GIT was created to solve

Git wasn't created because it was normal for projects rather; it emerged from a problem.

Prior to the development of Git, large projects were sharing source code through a product called BitKeeper (i.e., the Linux Kernel). While being offered free to some developers, it wasn't an open source product. This was sufficient for some time.

Then there came a time when BitKeeper's vendor changed their terms of service, thus restricting Linux developers from using BitKeeper at no cost anymore. This posed significant issues since Linux contributors across hundreds of geographical locations had amassed a massive amount of code.

So Linus decided that he was going to create a new invention for himself. He expressed dissatisfaction with pre-existing tools and so he created the first version of Git in 2005.

Git was coming into existence with an objective way to be developed:

- Speed
- Support for a distributed form of development (no longer requiring a central server)
- Reliability
- And most importantly, it had to be open source.

Until I created this project, I viewed Git as simply a means to manage the history of code through versions. After the fact, I see that Git is actually a solution to the challenges of managing collaboration among thousands of developers without the requirement to rely on a single company.

Additionally, Linus Torvalds (the founder of the Linux operating system) likely made Git open-source because of his prior experience with the Linux project. The Linux project was itself open-source, hence the logical decision to develop the tool that would be used to manage the project also as open source. If the tool had been created as proprietary software, it would most likely have faced the same issues again.

In summary, Git was created because developers wanted to have different and better solutions than available means, so the three factors that motivated the creation of Git were the need for freedom, control and speed that existed together with no other tool.

A2. The license — what it actually says

In my research of this project, I researched the Git license through the GNU General Public License version 2 (GPL v2).

When I first opened the license, I thought it was very long and hard to read because of all the legal language that was used. Once I started to break it down into sections and read the license slowly, I could understand the overall concept behind the license.

4 Free Software Freedoms

From what I can see, Free Software is based upon four (4) basic freedoms.

1) Freedom to use the software for any purpose you choose

2) Freedom to understand how the software works (which means that the source code has to be available)

3) Freedom to change the software

4) Freedom to share copies, including copies of your modified version of the software

All of these four freedoms are present in Git's GPL License; therefore, you can say that Git is free software, in the true sense of the word, and not just free to download.

Can Companies Commercialize Git Without Sharing Any Modifications?

This was somewhat confusing for me initially.

A business can take Git, change it, and sell it. However, they must share their changes because of the GPL, which is a copyleft license.

This means:

If you use this software and sell the modified version, you also must distribute your source code under the same license as the modified version you have created.

So it comes down to this:

You can make money selling the software

But you cannot create a version of the software that is not open source

I believe this has been done to support the openness of the software and to prevent anyone from taking what has been worked on by the open source community, and blocking it from being open.

Differences Between GPL and MIT Licenses (My Understanding):

Based on my review of both licenses, I have determined that:

With the GPL:

1)There are strict rules

2)You are required to share the changes that you have made

3)The software will continue to remain open to everyone

With the MIT:

1)It is very easy to understand and allows you the flexibility to modify and create a version that is not open source

2)No restrictions other than the copyright or patent rights

3)If I were going to create my own project, I think I would use the GPL, because:

I want to make sure that the work I have completed on my project remains open, and that no one can take the open source community's work and turn it into a private product.

However, I think if someone wants to build a project that spreads quickly, the MIT would be a good license to use.

Free Beer Vs Free As In Freedom

Until I learned of the existence of the two terms previously mentioned, I had never really understood completely what each of these terms meant.

"Free As In Free Beer" means you do not have to pay for the software; however, "Free As In Freedom" means that you are in control of the software.

A good example of freedom is Git.

While Git may be sold by a company that provides support or packages it in some way, the fundamental principle remains:

I can review the source code

I can modify it

I can distribute it

Therefore, when I think about free software, it makes no difference as to whether or not I have been charged for it. Each type of software has the same set of rights and freedoms for everyone.

So In Simple Terms

Having read the License, I believe that the GPL is not only a legal document, but more of a set of rules to safeguard that all open-source software has equal access and equity in its development and use.

Guaranteeing equal rights and access to any Open Source Software (i.e. everyone can use), Eliminating any possible owner of Open Source Software's ability to remove Open Source Software from the community, and Continues to be Open Source Software for any future enhancements of that particular Software.

Initially, I thought that a license was just a very formal piece of paper; however, as of now, I now realize that these types of documents actually outline how the user will be permitted to use, distribute, and use the software.

A3. The ethics of open source

I learned from this experience that open-source software isn't only about code and licensing, but more about people's attitudes towards knowledge-sharing and collaboration in ways they create. This part of the project led me to think more than the technical details did.

Is Open Source Good?

From my point of view, I can't answer that question with a simple "yes" or "no."

Pros To Open Source Software:

Open-source software opens up a lot of doors for people because they can share, learn from, and create products that are based on the code without any restrictions. Some examples include the Linux operating system and Git, which are tools being used by countless programmers throughout the world. If these tools were closed-source, then many people would not be able to take advantage of these tools.

Another benefit of open source software is that it provides transparency; therefore, any flaws or vulnerabilities in the code are openly stated, and users will have no excuse for using flawed or insecure software.

Cons To Open Source Software:

On the other hand, open-source software isn't always a good fit for all applications. Businesses invest a significant amount of time and money developing their products. If everything were to become open source, then businesses would risk losing their competitive advantage.

In addition, certain applications involve sensitive information or security issues; opening all of that data could pose a serious risk if not handled appropriately.

Is It Unethical for Companies to Make Money Through Open Source Software?

To be honest, I had trouble answering the question above and didn't know if it was ethical or unethical. For example:

Red Hat, Google, and other companies build on top of open-source software like Linux to sell products.

When I originally thought about this topic, I had an opinion that it was wrong for companies to profit from free software created by the community; this just seemed unfair.

However, then I thought about what these companies give back to the community:

1 – They put effort into improving & supporting open-source projects.

2 – They provide additional services and solutions using open-source projects.

3 – They maintain, host, and help scale open-source software projects.

So based on all of their contributions back to the open-source community, my belief is that it would not be unethical for companies like these to profit from the use of open-source products.

On the contrary, if any company were to build on only open-source projects without ever contributing back, I would feel that they were just abusing or taking advantage of the work of volunteers.

Constructions using pre-existing material are a continuing source of innovative technology development.

Software is one area where many of the systems being created are based upon prior knowledge (the case here is Git). For example:

* Many of the features in Git were originally developed through experience gained from previous systems of version control.

* Examples of systems of software that are currently in development (and that will be based upon Git or the Linux operating system) are myriad.

Developers, when they use previously constructed systems, have a far greater potential for creating innovative products than they do if they create their own systems from "scratch".

In my opinion, it is open-source software that promotes innovation by providing users with many of the initial or fundamental components they need to construct innovative types of applications.

Some would argue that having so many available "basic" programs would result in a lack of new ideas being produced by innovators. I would disagree! Having so many existing basic building blocks available allows developers to focus their resources toward the development of more sophisticated solutions that will address and resolve complicated issues!

If open-source software did not exist, then every developer would be forced to potentially reinvent many wheels each time they constructed a new product. This would create a tremendous bottleneck for innovative development in the future!

Part B — Linux Footprint (Unit 2)

Section B — Linux footprint

I have been using Git on a Linux system (basic Ubuntu installation) and learned where Git exists on the system and how to use it.

Before this section I had only ever been using Git commands on projects – I never found out where Git was installed or how it worked internally in a Linux system. This section helped explain those things to me.

How to install Git:

On Linux it's really easy to install Git.

Using the Apt package manager on my system I ran:

sudo apt update

sudo apt install git

Git is available by default in most distributions of Linux, so there is no need to manually download and compile Git from source.

I have also read that if you're looking for the most up-to-date version of Git, a user can compile Git from source. However for most common users, the package manager (apt) is sufficient.

Key locations for Git:

After installing Git I looked at what directories Git files are stored in:

Where is Git file(s) located?

Add binary location: (which git) → "/usr/bin/git".

Since the main executable for Git is located at "/usr/bin" this is where many other systems commands are also located (like ls, cp, rm, etc.).

Users and Groups

The Git program does not run as a separate system user like services do (e.g., web servers).

Instead, it runs as the currently logged-in user.

For example:

If I run Git, it runs as me.

This is an important consideration when looking at security implications:

Git will only have access to the files the current user has permission to access.

There are no root privileges required to run git (except for installing it).

Therefore, it is safer to use Git than to run a service as a higher-privilege user.

Service Management

Git is not a background service.

That means that we cannot use systemctl to control it, e.g.,

"systemctl start git."

Git only runs when one of the following commands is executed:

"git clone,"

"git commit,"

"git push."

I was surprised to see that Git was not a service like every other program on my Linux system. It's completely different.

Update Model

Updates for Git are done using your Linux package manager.
For example:
Using the apt package manager:
$ sudo apt update
$ sudo apt upgrade

Updates (bug fixes, security patches) from the open-source community are:
Added to the git repository,
Packaged by the Linux distro maintainers,
Distributed via your system using apt.

This clearly demonstrates how open-source updates are distributed via an organized flow:
Developer → Open-source repository → Linux distribution → User system.

While doing this section, I realized:

- Git is very lightweight
- It does not depend on background services
- It is tightly integrated with the Linux file system
- Most of its work happens inside project folders

Part C — The FOSS Ecosystem (Units 3 & 4)

As I was working on this project and investigating what open-source software actually is and how it interacts with one another, I realized that no open-source software (OSS) works entirely independently. Everything connects to each other in some way, including Git.

Prior to this, my assumption was that Git was simply a tool used for version control and that was it; however, as I continued to do more research, I discovered that Git is much larger than that; it exists in a larger ecosystem of tools and communities.

Below, I will list out the things Git relies on in order to operate.

Git is primarily written in C. Thus, in order for Git to run, it also requires several basic system tools and libraries.

Below are several of those important elements:

- GCC - used to compile Git from source.
- Standard C libraries - required for basic functionality.
- Shell Tools (like Bash) - many operations that Git performs internally use the Bash shell.

When I originally installed Git by using apt, I did not see these dependencies because everything was done automatically. However, after learning more, I found that Git depends heavily on the existence of the Linux environment.

Thus, while Git is fairly easy for users to understand and use, there is a high degree of complexity behind the scenes, since it relies on many small tools working in conjunction with one another.

How Git Has Enabled Other Tools

This section is what I found to be the most fascinating.

Relation to LAMP Stack / Web
The LAMP stack (Linux, Apache, MySQL, PHP) does not include Git. However, Git indirectly supports the LAMP stack.

An example: developers use Git to manage their web application code, which is then uploaded to a web server using a web server program like Apache HTTP Server. The database program used on the web server when the application code is uploaded is usually MySQL.

Because Git is used during the development phase, it exists in front of deployment.

At first, I thought that Git was used for hosting websites; instead, Git is primarily used to manage the source code that is used to create a website.

Community and Governance
Git is a robust open-source community, which includes contributors globally who participate in maintaining the project. The project was started by Linus Torvalds. Many other contributors (different than Torvalds) now contribute to the project (I discovered this fact when researching related information on the web about Git).

Some Available Resources:
• Mailing Lists
• Discussion Forums
• Online Documentation
• Developer Groups/Communities on GitHub

I also learned that the development process for Git happens in a well-organized and controlled manner.

• Changes are sent to a review group for approval before they may be applied to the main project branch.
• Not anyone can change the main project directly, only authorized contributors.

Even in an open-source environment, there is an ongoing system of control and organization.

My Observations
During this process of learning about Git I learned that:

• Git relates to a broad range of applications.

Part D — Open Source vs Proprietary (Critical Analysis)

I will be investigating the differences between GIT (Open Source) and Perforce Helix Core. Perforce Helix Core is a proprietary and widely used version control solution used by many companies.

I had thought that Open Source was always a better product than Proprietary. After conducting my investigations and comparing them, I've come to the conclusion that each of them has benefits to offer.

1. Price

Open Source vs Proprietary

GIT (Open Source)

GIT is completely free. No payments are required to download or use the software. That is one of the primary reasons for its increase in popularity.

Perforce (Proprietary)

Perforce has a basic free version that only allows for a limited number of users. Large teams will have to pay to receive a license for the software.

My Thoughts

For Students and small teams GIT is definitely the better choice because price is important to them. However, price may not be the primary reason for making a choice for a large company.

2. Security (Who is able to audit the source code)

Open Source vs Proprietary

GIT

Being open-source allows anyone to look at the code and verify whether there are bugs or other security vulnerabilities. It also creates an environment where people can see the inner workings of the product and you can be more confident that it is a secure product.

Perforce

The code is proprietary and only available for the use of the Perforce Company.

My Thoughts

Open source feels much more reputable due to the ability of many people looking at the code, however, many times companies develop a dedicated security team and therefore closed source does not equate to a lack of security.

Dimension	Open Source	Proprietary Alternative
Cost	Free to use. No cost at all.	Free for small use, but paid for bigger teams.
Security (who can audit the code?)	Code is open, anyone can check it.	Code is closed, only company can check it
Support and reliability	Community help, no official support	Has official support and services.
Freedom to modify	Can modify code, but must share changes	Cannot modify, only use as it is.
Community vs corporate control	Managed by community, many contributors.	Controlled by a single company.
Your overall verdict	Best for learning, flexibility, and open use.	Better for companies needing support and control.

4. Shell Script Tasks

Script 1 — System Identity Report (Unit 1 & 2)

Write a script that introduces the Linux system like a welcome screen. It should display:

- The Linux distribution name and kernel version
- The current logged-in user and their home directory
- System uptime and current date/time
- A message stating which open-source license covers the OS

Concepts to use: variables, echo, command substitution ($()), basic output formatting.

```
# Getting Linux distribution name
distro=$(cat /etc/os-release | grep PRETTY_NAME | cut -d= -f2 | tr -d '"')
# Getting kernel version
kernel=$(uname -r)
# Getting current user
user=$(whoami)
# Getting home directory
home_dir=$HOME
# Getting system uptime
uptime_info=$(uptime -p)
# Getting current date and time
current_time=$(date)
# Assuming Linux uses GPL license (general statement)
license="GNU General Public License (GPL)"
# Displaying the information
echo "----------------------------------------"
echo "   Welcome to Your Linux System 🚀"
echo "----------------------------------------"
echo "Distribution : $distro"
echo "Kernel       : $kernel"
echo ""
echo "User         : $user"
echo "Home Dir     : $home_dir"
echo ""
echo "Uptime       : $uptime_info"
echo "Date & Time  : $current_time"
echo ""
echo "License Info : This system is based on $license"
echo "----------------------------------------"
echo "Have a great day! 😊"
echo "----------------------------------------"
```

Output: -

Output: 7 ms | 3:

```
----------------------------------------
   Welcome to Your Linux System 🚀
----------------------------------------
Distribution : Ubuntu 24.04.3 LTS
Kernel       : 6.1.159-181.297.amzn2023.x86_64

User         :
Home Dir     : /tmp

Uptime       : up 6 hours, 24 minutes
Date & Time  : Tue Mar 31 07:57:31 AM UTC 2026

License Info : This system is based on GNU General Public License (GPL)
----------------------------------------
Have a great day! 😊
----------------------------------------
```

Script 2 — FOSS Package Inspector (Unit 2)

```
#!/bin/bash
# Ask for package name
read -p "Enter package name: " PACKAGE
# Check if installed
if dpkg -l | grep -w $PACKAGE > /dev/null
then
    echo "$PACKAGE is installed"
    dpkg -s $PACKAGE | grep -E "Version|Description"
else
    echo "$PACKAGE is NOT installed"
fi
# Case statement for description
case $PACKAGE in
    git) echo "Git: version control system" ;;
    apache2) echo "Apache: web server" ;;
    mysql) echo "MySQL: database system" ;;
    vlc) echo "VLC: media player" ;;
    *) echo "No info available" ;;
esac
```

Script 3 — Disk and Permission Auditor (Unit 2)

```
#!/bin/bash
# List of directories
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")
echo "Directory Audit Report"
echo "----------------------"
# Loop through directories
for DIR in "${DIRS[@]}"
do
    if [ -d "$DIR" ]; then
        # Get permissions, owner, group
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')

        # Get size
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)

        echo "$DIR => $PERMS | Size: $SIZE"
    else
        echo "$DIR => Not found"
    fi
done
```

Output

Script 4 — Log File Analyzer (Units 2 & 5)

```bash
#!/bin/bash
# Get inputs
LOGFILE=$1
KEYWORD=${2:-"error"}
COUNT=0
# Check if file exists
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File not found."
    exit 1
fi

# Check if file is empty (retry once)
if [ ! -s "$LOGFILE" ]; then
    echo "File is empty. Trying again..."
    sleep 2
fi
# Read file line by line
while IFS= read -r LINE
do
    if echo "$LINE" | grep -iq "$KEYWORD"
    then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"
# Output result
echo "Keyword '$KEYWORD' found $COUNT times"
# Show last 5 matching lines
echo "Last 5 matches:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

Script 5 — The Open Source Manifesto Generator (Unit 5)

```bash
#!/bin/bash
# Script 5: Open Source Manifesto Generator
echo "Answer three questions to generate your manifesto."
echo ""
# Taking user input
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD
# Getting date
DATE=$(date '+%d %B %Y')
# Output file name
OUTPUT="manifesto_$(whoami).txt"
# Writing manifesto to file
echo "Open Source Manifesto - $DATE" > $OUTPUT
echo "" >> $OUTPUT
echo "I use $TOOL in my daily work, and it shows how powerful open-source can be." >> $OUTPUT
echo "For me, freedom means $FREEDOM, especially in software and technology." >> $OUTPUT
echo "In the future, I would like to build $BUILD and share it openly with others." >> $OUTPUT
echo "I believe knowledge should be shared, not restricted." >> $OUTPUT
# Note: You can create an alias like:
# alias manifesto='./script5.sh'
# Display result
echo ""
echo "Manifesto saved to $OUTPUT"
echo ""
cat $OUTPUT
```

5. Project Timeline

This timeline is a guide, not a rigid rule. Some students will move faster on the technical parts; others will spend more time on the philosophical sections. Adjust based on your strengths — but do not leave the scripts to the final stretch.

Day	Focus	Tasks	Deliverable
1	**Setup**	Choose software. Research its origin story. Read the license text. Set up your Linux environment (VM or lab system). Install your chosen software.	***Software selected & installed Choice confirmed***
2	**Philosophy**	Write Part A1 — the origin story. Take notes on what problem the software solved and for whom.	***Draft A1***
3	**License & Ethics**	Write Part A2 — license analysis. Write Part A3 — ethical reflection. These sections need your own thinking, not summaries.	***Draft A2 + A3***
4	**Linux Footprint**	Install software on Linux. Document directories, user, permissions. Write Part B.	***Draft Part B + screenshots***
5	**Scripts 1 & 2**	Write and test Script 1 (System Identity) and Script 2 (Package Inspector). Add them to your report with explanations.	***Scripts 1–2 working***
6	**Scripts 3 & 4**	Write and test Script 3 (Disk Auditor) and Script 4 (Log Analyzer). Test with a real log file.	***Scripts 3–4 working***
7	**Ecosystem**	Write Part C — FOSS ecosystem map. Research dependencies and community.	***Draft Part C***
8	**Comparison & Script 5**	Write Part D — comparison table and verdict. Write and test Script 5 (Manifesto Generator).	***Draft Part D***
9	**Report polish**	Compile full report. Write introduction and conclusion. Add screenshots. Check page count (12–16 pages).	***Script 5***
10	**Submission**	Final review. Submit report + 5 .sh files. Be ready to explain any script in a short viva if asked.	***Final submission***

6. Conclusion

The project gave me an opportunity to learn about open-source software and its advantages. Before I completed the report, I had used various tools (like Git) without thinking about how they were created or why they are considered "open source." Through working on the report, I found out that open source is not just free software, but a system of sharing, collaborating on, and providing control of software for people.

As I researched Git, I learned about how Git was developed because of a desire for the community to share and collaborate on code and ensure that they would always have access to that code through GPL licensing. I also learned the importance of maintaining projects through the contributions of the community.

The Linux portion of the project helped me gain more insight into the functions of Git. I discovered where Git can be installed, how it interacts inside the operating system, and how commands are executed. It was my first experience writing shell scripts; while I made a few mistakes when I first started writing them, once I tested and fixed those errors, I became more adept at scripting.

In the Ecosystem section of the project, I learned that no software operates in isolation; many tools and platform integrate with Git, and Git is a crucial component to modern programming.Comparing the two types of software allowed me to further realize that open-source software isn’t better in all cases and proprietary software has its advantages as well.

Through this project I’ve gained technical and cognitive growth, but also gained an appreciation for the value of open source software in supporting further innovation because it allows for sharing of ideas through collaboration.

In the future I hope to either participate as a contributor on open source projects, or create something that I could share.